

Concepts of Operating Systems And Linux Operating Systems

Trainer: Kiran Jaybhave

Classification of OS

- OS can be categorized based on the target system (computers).
  - Mainframe systems
  - Desktop systems
  - Multi-processor (Parallel) systems
  - Distributed systems
  - Hand-held systems
  - Real-time systems

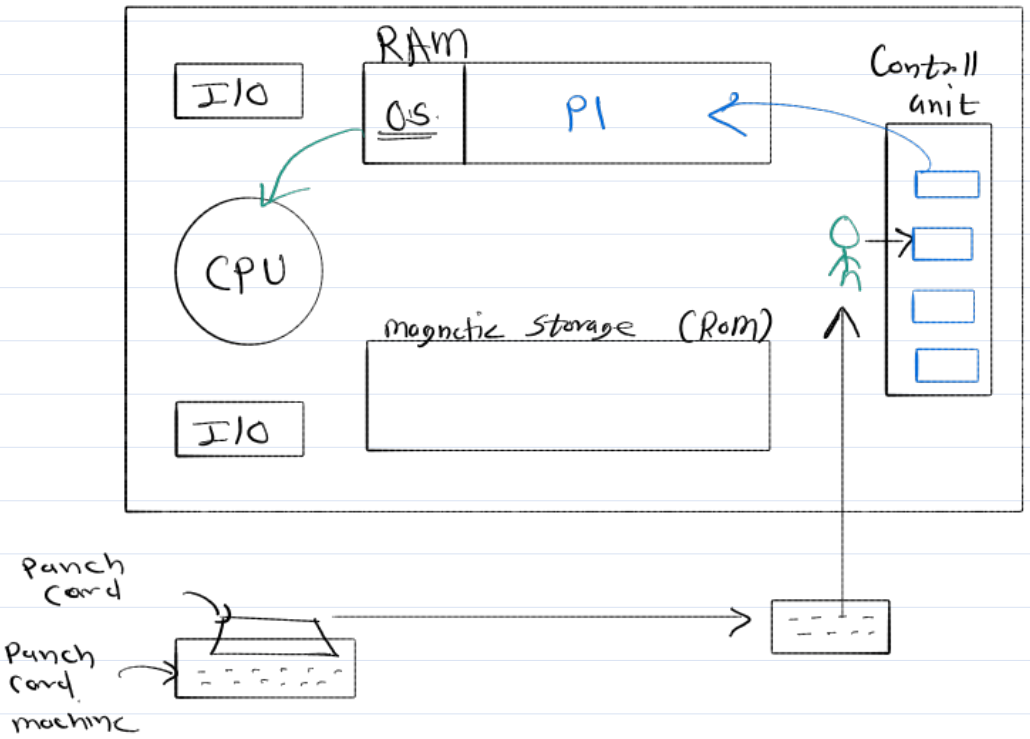
SUNBEAM

Mainframe systems

- Examples: UNIX and its flavours, IBM-360, etc.
- **Resident Monitor**
  - Early (oldest) OS resides in memory and monitor execution of the programs. If it fails, error is reported.
  - OS provides hardware interfacing that can be reused by all the programs.
- **Batch Systems**
  - The batch/group of similar programs is loaded in the computer, from which OS loads one program in the memory and execute it.
  - The programs are executed one after another.
  - In this case, if any process is performing IO, CPU will wait for that process and hence not utilized efficiently.

SUNBEAM

*Mainframe System



① Resident monitor

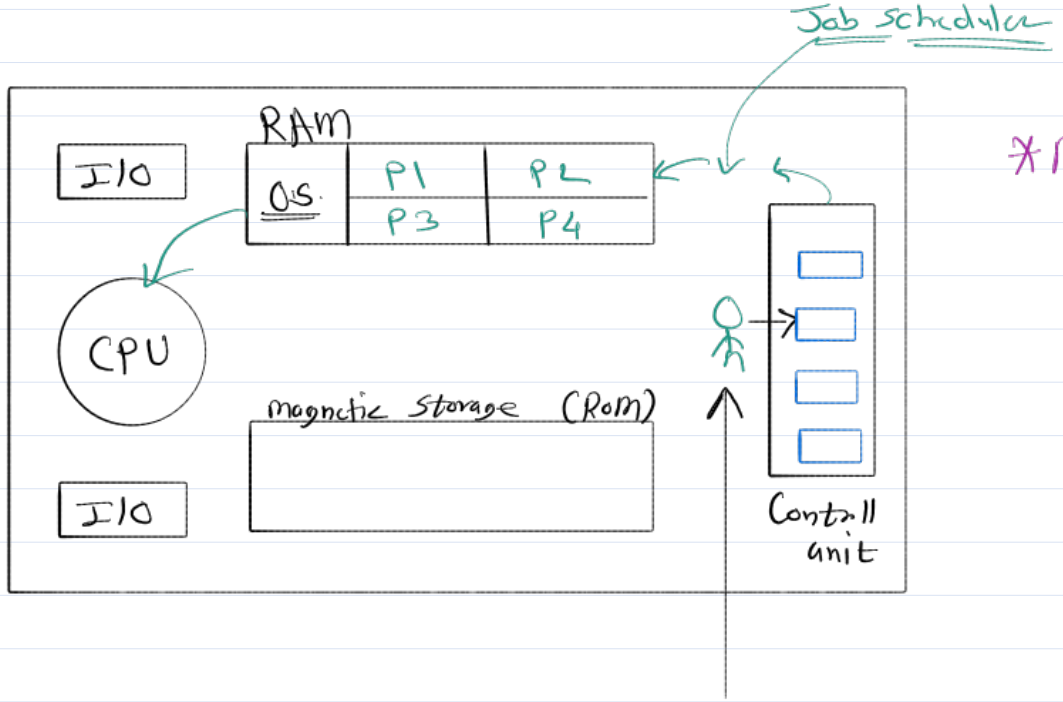
② Batch system

Multi-Programming

- In multi-programming systems, multiple program can be loaded in the memory.
- The number of program that can be loaded in the memory at the same time, is called as "**degree of multi-programming**".
- In these systems, if one of the process is performing IO, CPU can continue execution of another program.
- This will increase CPU utilization.
- Each process will spend some time for CPU computation (CPU burst) and some time for IO (IO burst).
  - If CPU burst > IO burst, then process is called as "**CPU bound**".
  - If IO burst > CPU burst, then process is called as "**IO bound**".
- To efficiently utilize CPU, a good mix of CPU bound and IO bound processes should be loaded into memory.
- This task is performed by an unit of OS called as "**Job scheduler**" OR "**Long term scheduler**".
- If multiple programs are loaded into the RAM by job scheduler, then one of process need to be executed (dispatched) on the CPU.
- This selection is done by another unit of OS called as "**CPU scheduler**" OR "**Short term scheduler**".

SUNBEAM

* Multiprograming



* Multiprograming

- Multiple program loaded in RAM
  - CPU burst : Man time spend on CPU
  - I/O burst : Man time spend on I/O

- CPU burst > I/O burst : CPU bound process
- I/O burst > CPU burst : I/O bound process

- Mix type of process load on RAM
- Num of process loaded on RAM is known as degree of Multiprograming.

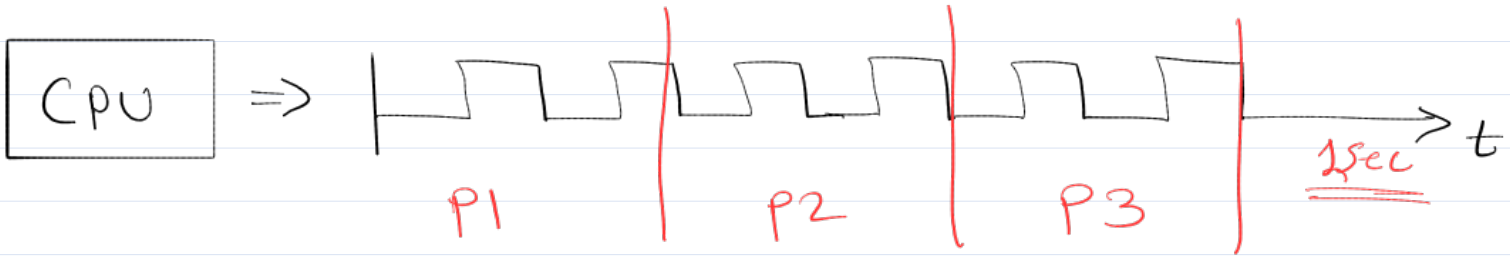
- **Multi-tasking OR time-sharing**
  - CPU time is shared among multiple processes in the main memory is called as "**multi-tasking**".
  - In such system, a small amount of CPU time is given to each process repeatedly, so that response time for any process < 1 sec.
  - With this mechanism, multiple tasks (ready for execution) can execute concurrently.
  - There are two types of multi-tasking:
    - **Process based multitasking**:
      - Multiple independent processes are executing concurrently.
      - Processes running on multiple processors called as "**multi-processing**".
    - **Thread based multi-tasking OR multi-threading:**
      - Multiple parts/functions in a process are executing concurrently.

SUNBEAM

* Time sharing System :

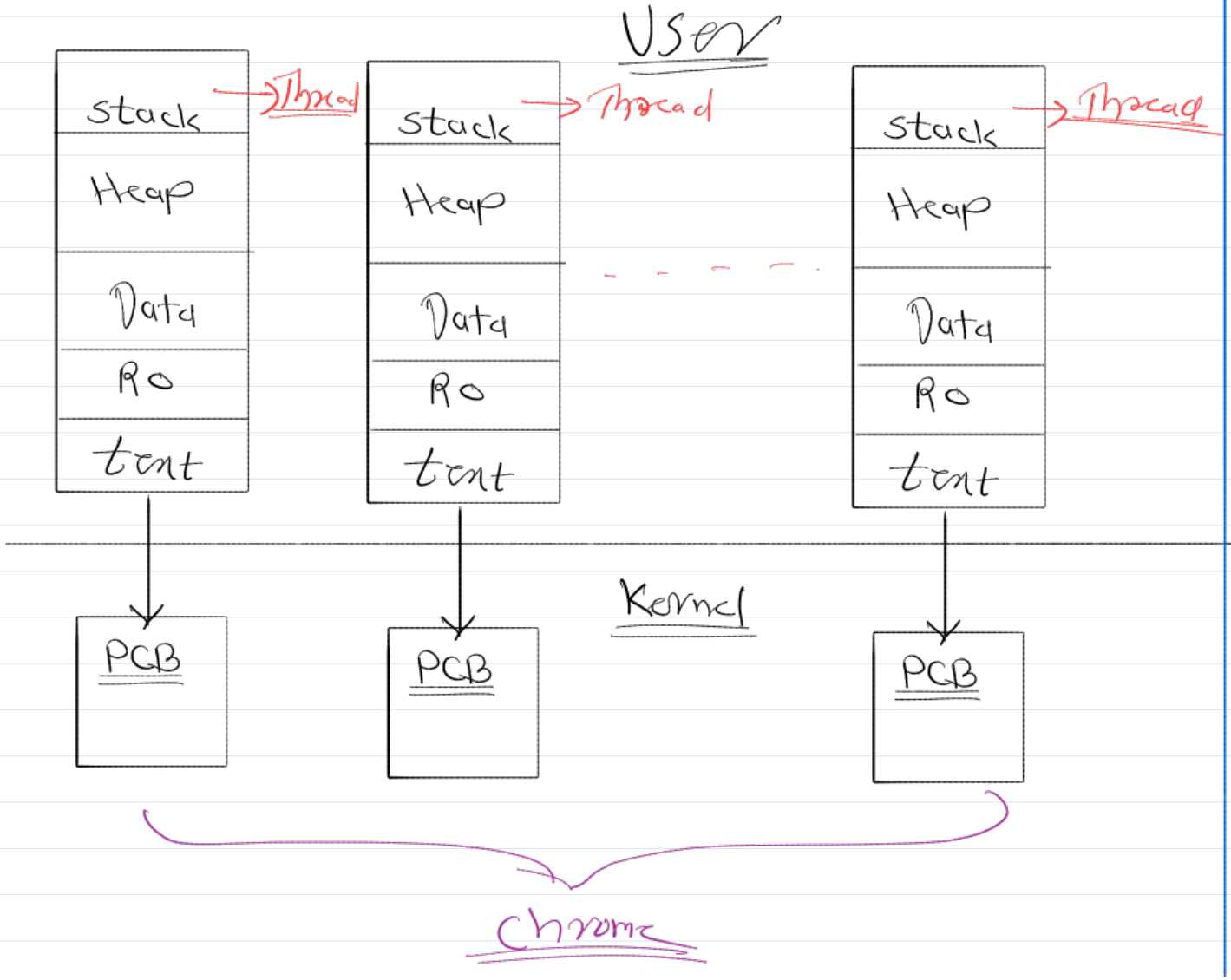
T.Q (20)

1sec → 1000

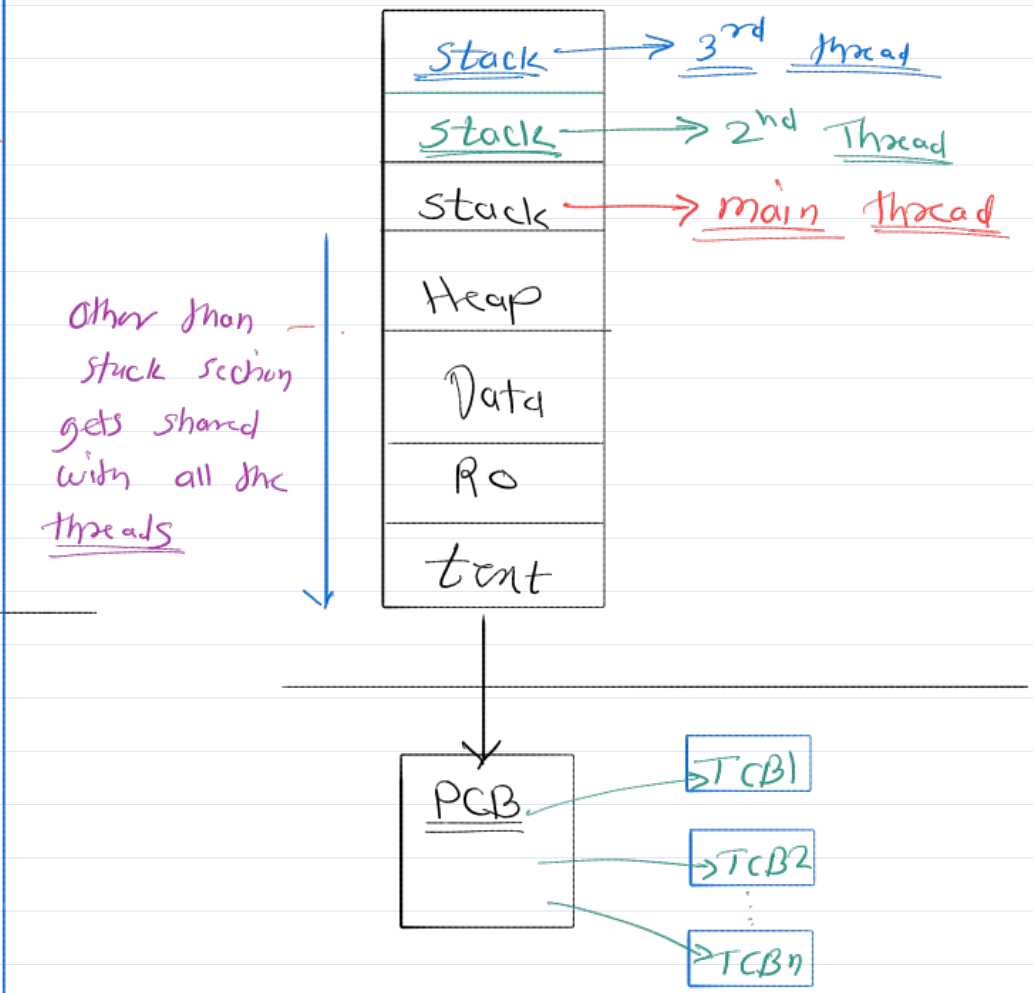


→ Process based multitasking
→ Thread based multitasking.

Processed based M.T.



Thread based M.T.



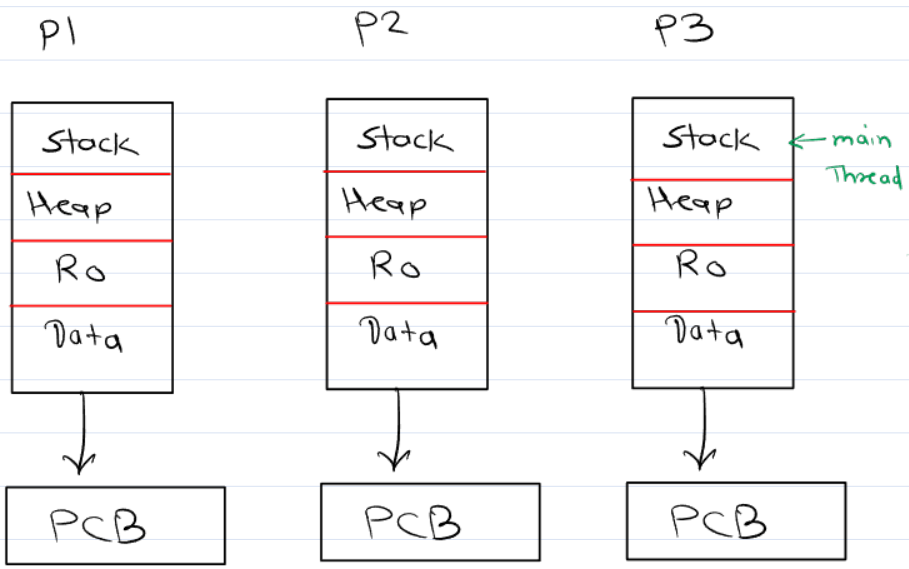
Thread : light weight process.

* Time sharing system : CPU time share with all Running process.

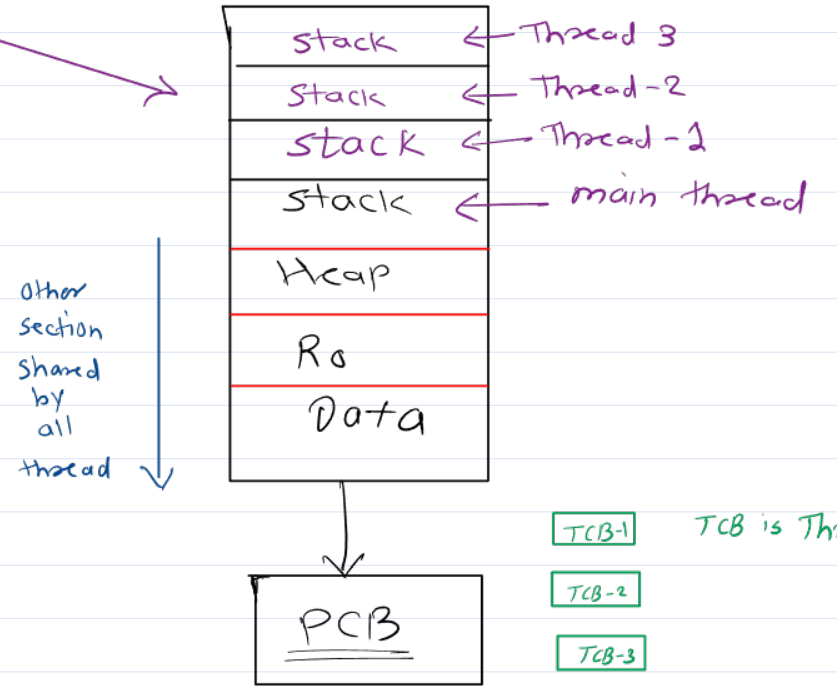
: Response time < 1 sec.

Process based Multi-tasking

Thread based multi-tasking.



→ For process all section are created in user space + PCB in created in Kernel space.



i.e - OS Create thread means Create one more stacks section
- Thread is light weight process
- Other section shared by all thread.

TCB is Thread Controll block

- For thread only stack section is created in user space & TCB is created in kernel space
- Remaining section of user space are shared with parent process

Thread

- **Thread concept**
  - Threads are used to execute multiple tasks concurrently in the same program/process.
  - Thread is a light-weight process.
    - For each thread new control block and stack is created. Other sections (text, data, heap, ...) are shared with the parent process.
    - Inter-thread communication is much faster than inter-process communication.
    - Context switch between two threads in the same process is faster.
  - Thread stack is used to create function activation records of the functions called/executed by the thread.
- **Process vs Thread**
  - In modern OS, process is a container holding resources required for execution, while thread is unit of execution/scheduling.
  - Process holds resources like memory, open files, IPC (e.g. signal table, shared memory, pipe, etc.).
  - PCB contains resources information like pid, exit status, open files, signals/ipc, memory info, etc.
  - CPU time is allocated to the threads. Thread is unit of execution.
  - TCB contains execution information like tid, scheduling info (priority, sched algo, time left, ...), Execution context, Kernel stack, etc.
  - terminal> ps -e -o pid,nlwp,cmd
  - terminal> ps -e -m -o pid,tid,nlwp

SUNBEAM

- **main thread**
  - For each process one thread is created by default called as main thread.
  - The main thread executes entry-point function of the process. The main thread use the process stack.
  - When main thread is terminated, the process is terminated.
  - When a process is terminated, all threads in the process are terminated.

SUNBEAM

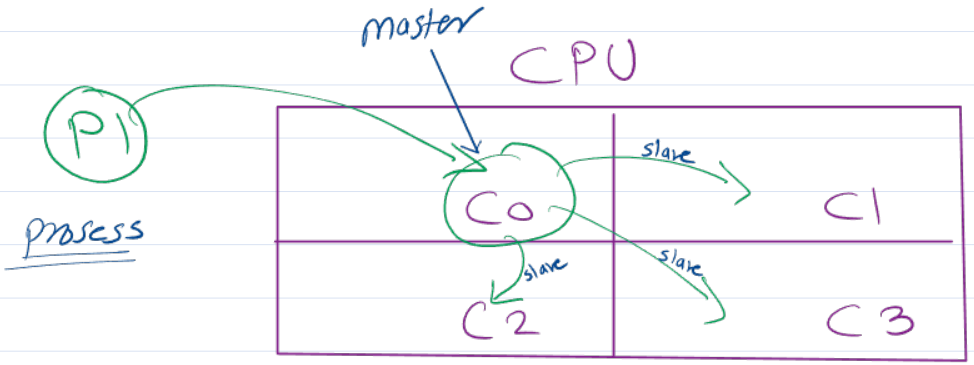
Multiprocessor systems

- The systems in which multiple processors are connected in a close circuit is called as "**multiprocessor computer".**
- The programs/OS take advantage of multiple processors in the computer are called as " Multi-procssing " programs/OS.
  - Windows Vista : First Windows OS designed for multi-processing.
  - Linux 2.5+ : Linux started supporting multi-processing.
  - terminal> uname -a
- Since multiple tasks can be executed on these processors simultaneously, such systems are also called as **"parallel systems".**
- Parallel systems have more throughput (Number of tasks done in unit time).
- There are two types of multiprocessor systems:
  - Asymmetric Multi-processing
  - Symmetric Multi-processing

SUNBEAM

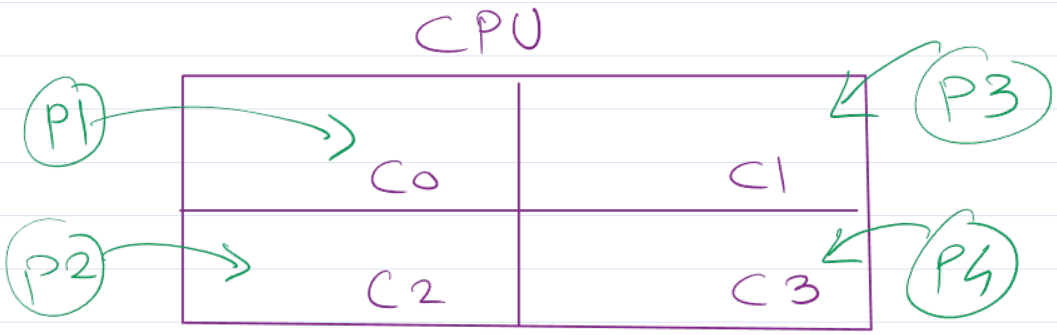
* Multiprocessor

- Asymmtric m.p
- symmtric m.p



Asymmtric Multi-processor

(All core execute single task)



Symmtric Multi-processor

(each core execute diff. task)

- **Asymmetric Multi-processing**
  - OS treats one of the processor as master processor and schedule task for it.
  - The task is in turn divided into smaller tasks and get them done from other processors.

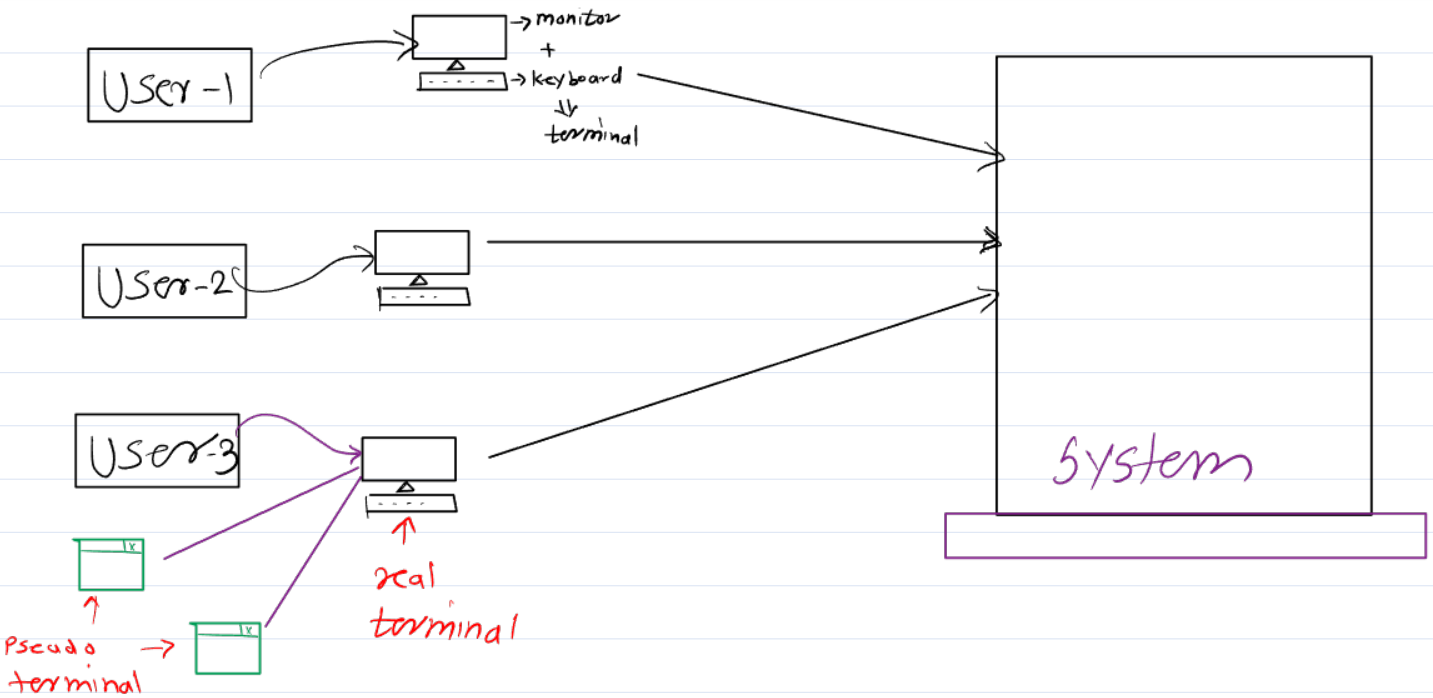
- **Symmetric Multi-processing**
  - OS considers all processors at same level and schedule tasks on each processor individually.
  - All modern desktop systems are SMP.

SUNBEAM

- **Multi-user**
  - Multiple users can execute multiple tasks concurrently on the same systems.
  - e.g. IBM 360, UNIX, Windows Servers, etc.
  - Each user can access system via different terminal.
  - There are many UNIX commands to track users and terminals.
    - **tty** (teletype) : It prints the name of the current terminal.
    - **who** : Information about currently logged in users, system boot time, runlevel , processes,…
    - **who am i** : Gives you the name of the current user, the terminal they are logged in at, the date and time when they logged in.
    - **whoami** : It gives username of the current user
    - **w** : Displays the users.

SUNBEAM

*multi user System



Commands : who :
tty :
who am i :
w :

Desktop systems

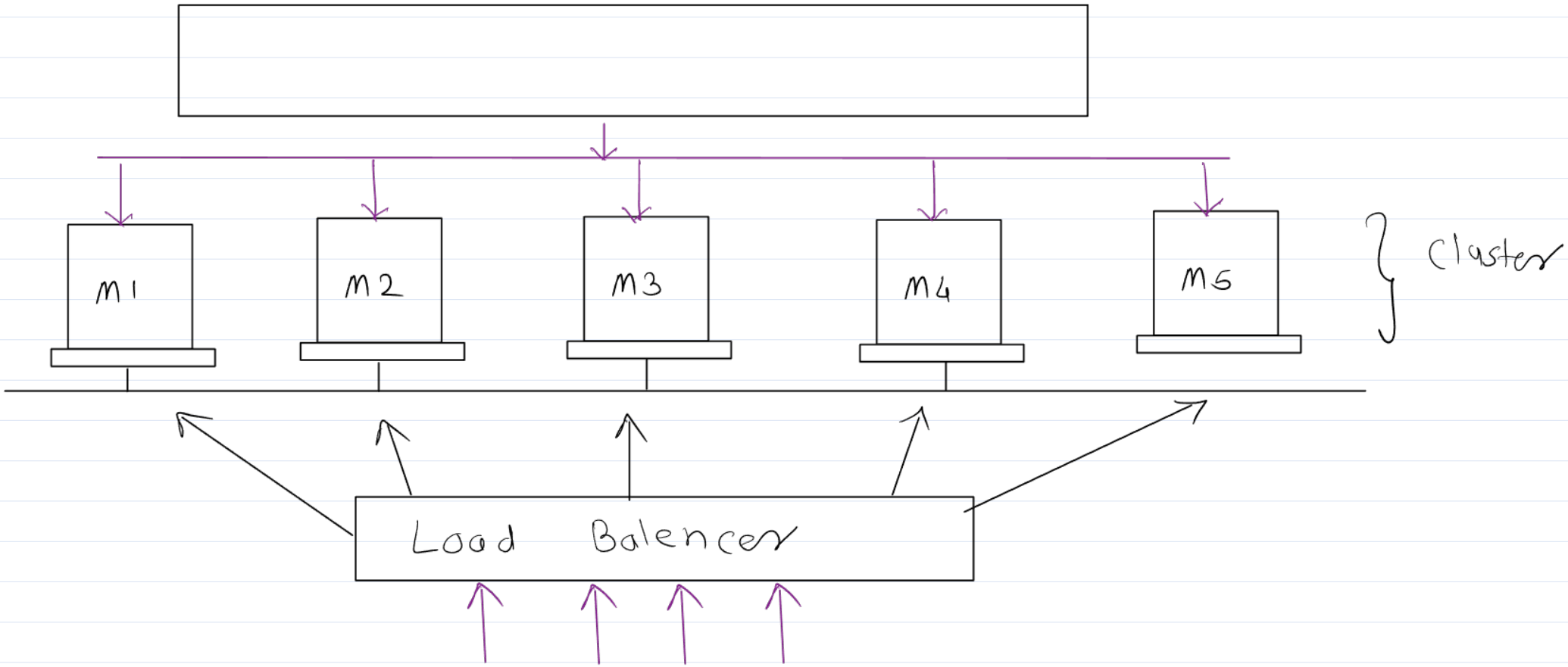
- Personal computers -- desktop and laptops
- User convenience and Responsiveness
- Examples: Windows, Mac, Linux, few UNIX, ...

SUNBEAM

- **Distributed systems**
  - Multiple computers connected together in a close network is called as "distributed system".
  - Its advantages are high availability (24x7), high scalability (many clients, huge data), fault tolerance (any computer may fail).
  - The requests are redirected to the computer having less load using "load balancing" techniques.
  - The set of computers connected together for a certain task is called as "cluster".
  - Examples: Linux

SUNBEAM

* Distributed System



- **Handheld systems**
  - OS installed on handheld devices like mobiles, PDAs, iPODs, etc.
  - Challenges:
    - Small screen size
    - Low end processors
    - Less RAM size
    - Battery powered
  - Examples: Symbian, iOS, Linux, PalmOS, WindowsCE, etc.

SUNBEAM

- **Real-time Systems**
  - The OS in which accuracy of results depends on accuracy of the computation as well as time duration in which results are produced, is called as "RTOS".
  - If results are not produced within certain time (deadline), catastrophic effects may occur.
  - These OS ensure that tasks will be completed in a definite time duration.
  - Time from the arrival of interrupt till begin handling of the interrupt is called as "Interrupt Latency".
  - RTOS have very small and fixed interrupt latencies.
  - RTOS Examples: uC-OS, VxWorks, pSOS, RTLinux, FreeRTOS, etc

SUNBEAM

➢Process Management

- When we say an OS does process management it means **an OS is responsible for process creation, to provide environment for an execution of a process, resource allocation, scheduling, resources management, inter process communication, process coordination, and terminate the process.**

Q. What is a Program?

❑User view:

- Program is a finite set of instructions written in any programming language given to the machine to do specific task.

❑System view:

- Program is an executable file in HDD which divided logically into sections like exe header, bss section, data section, rodata section, code section, symbol table.

SUNBEAM

Q. What is a Process?

❑ **User view:**

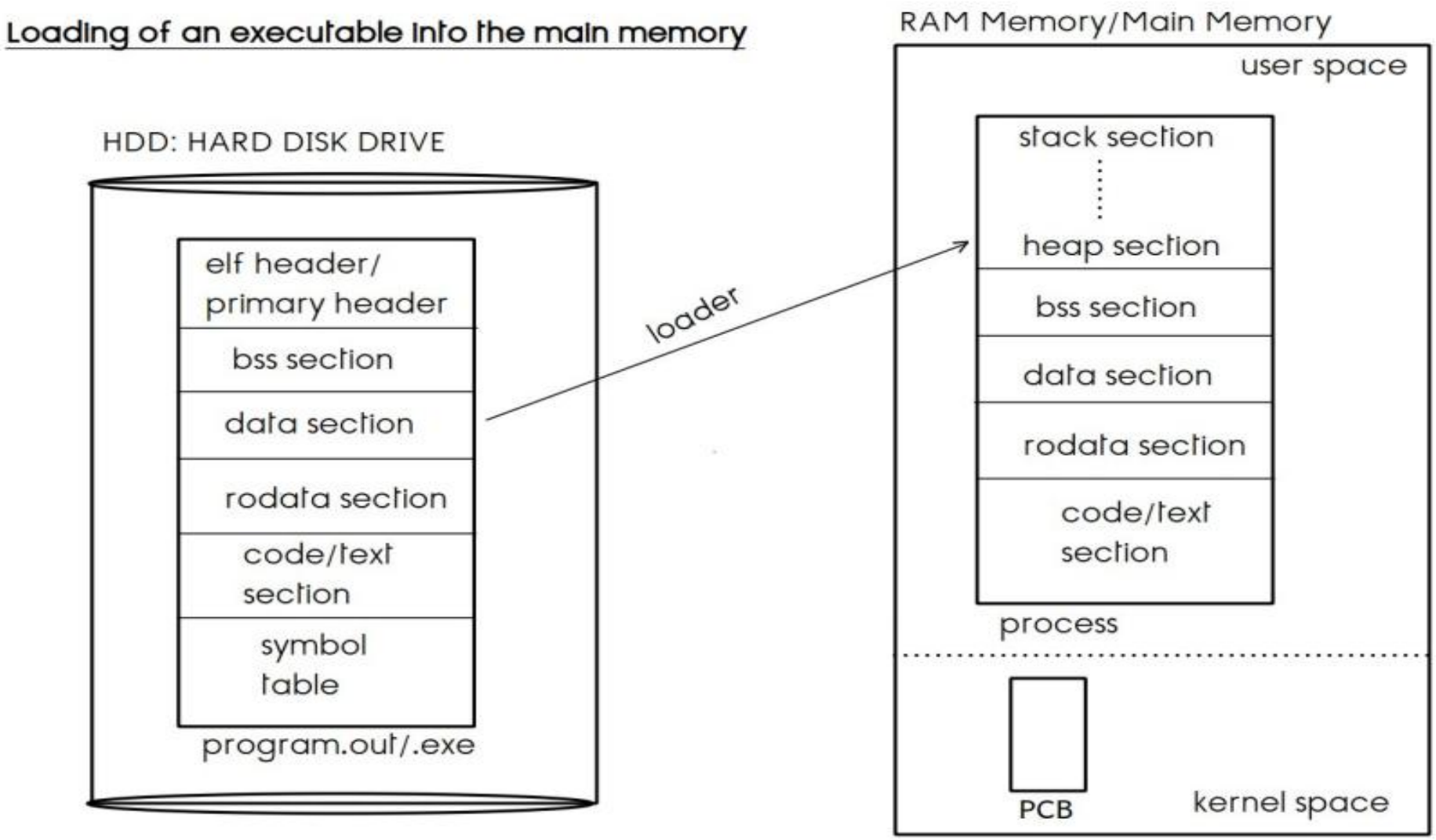
- Program in execution is called as **a process.**
- Running program is called as **a process.**
- When a program gets loaded into the main memory it is referred as **a process.**
- Running instance of a program is referred as **a process**.

❑ **System view:**

- Process is a program loaded into the main memory which has got PCB into the main memory inside kernel space and program itself into the main memory inside user space has **got bss section, rodata section, code section, and two new sections gets added for the process .**

- **stack section:** contains function activation records of called functions.
- **heap section:** dynamically allocated memory

SUNBEAM


Loading of an executable into the main memory
HDD: HARD DISK DRIVE
elf header/ primary header
bss section
data section
rodata section
code/text section
symbol table
program.out/.exe
loader
RAM Memory/Main Memory
user space
stack section
heap section
bss section
data section
rodata section
code/text section
process
PCB
kernel space

- As a kernel, core program of an a OS runs continuously into the main memory, part of the main memory which is occupied by the kernel **referred as kernel space** and whichever part is left **is referred as an user space**, so **main memory is divided logically into two parts: kernel space & user space.**
- **User programs gets loaded into the user space only.**
- When we execute a program or upon submission of a process very first one structure gets created into the main memory inside kernel space by an OS in which all **the information which is required to control an execution of that process** can be kept, this structure is referred as a **PCB.**
- **PCB : Process Control Block, is also called as a Process Descriptor.**
- Per process one PCB gets created and PCB remains inside the main memory throughout an execution of a program, upon exit PCB gets destroyed from the main memory.

SUNBEAM

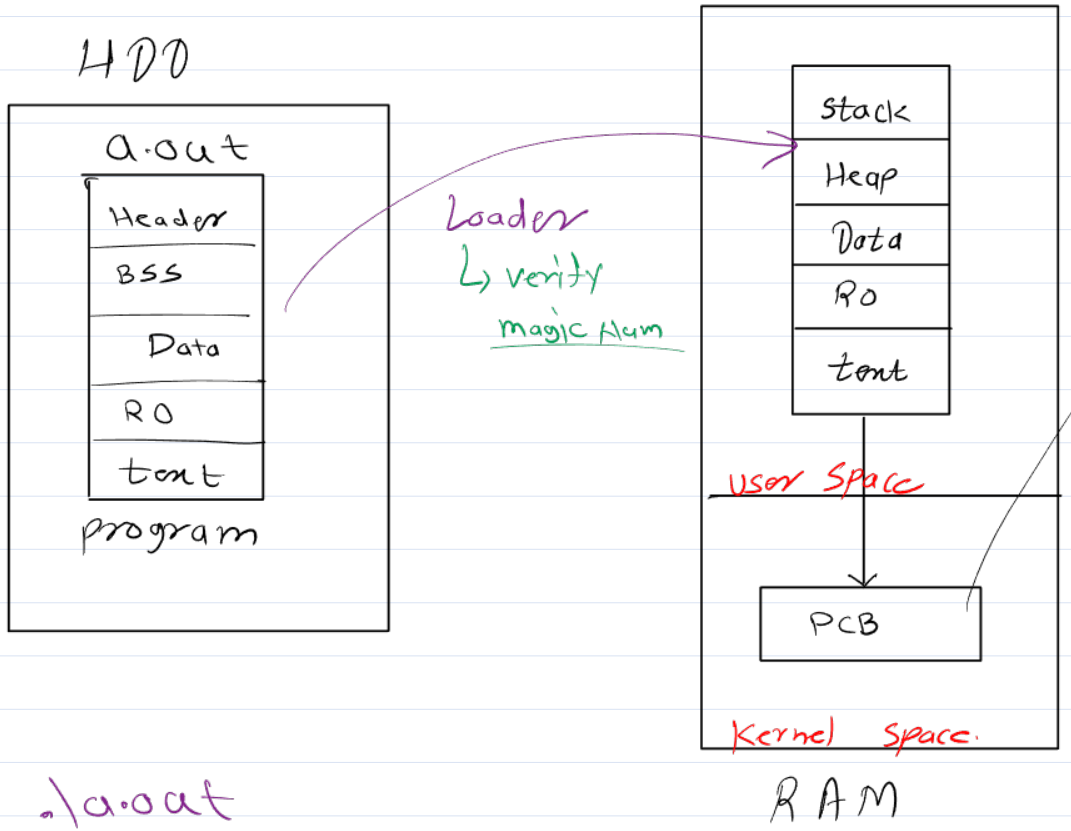
❖PCB mainly contains:

- PID : Process ID
- PPID : Parent Processes ID
- PC : Program Counter
- Execution context
- Kernel stack
- Exit status

SUNBEAM

- CPU sched information, memory management information, information about resources allocated for that process, execution context etc...

Process Management



./a.out

PCB / task_struct

PCB Contains

1. Pid
2. PPid
3. exit status
4. Scheduling info : CPU scheduling info (sched Algo)
   Quatan time. (ipc)
5. Memory info : Segmentaion / Paging
   : RAM address
6. Files info : open file info
   : Current Patch.
7. Exccation Content
8. Kernel stack :

➢To keep track on all running programs, an OS maintains few data structures referred **as kernel data structures:**

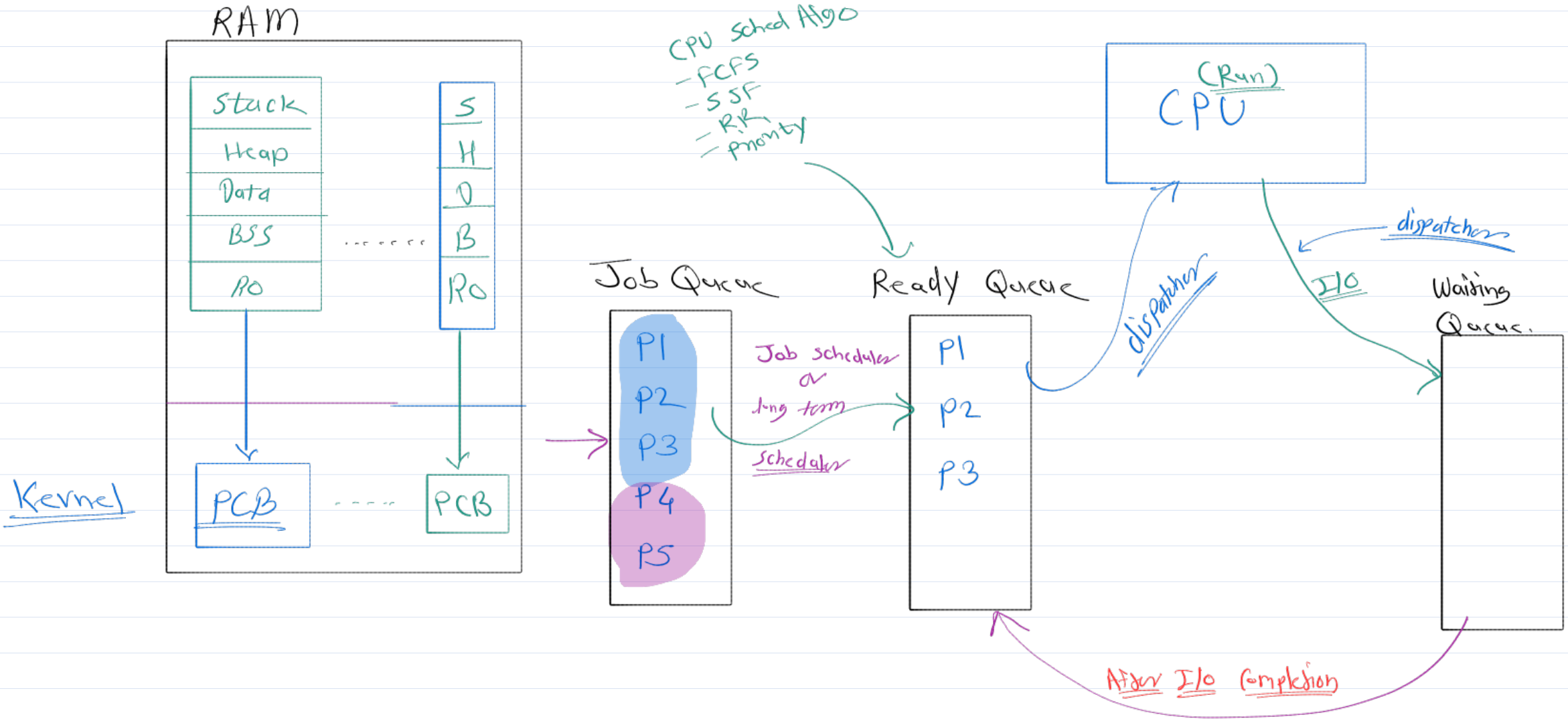
1. **Job queue:** it contains list of PCB's of all submitted processes.
2. **Ready queue:** it contains list of PCB's of processes which are in the main memory and waiting for the CPU time.
3. **Waiting queue:** it contains list of PCB's of processes which are requesting for that particular device.

➢**Scheduler :**

1. **Job Scheduler/Long Term Scheduler:** it is a system program which selects/schedules jobs/processes from job queue to **load them onto the ready queue**.
2. **CPU Scheduler/Short Term Scheduler:** it is a system program which selects/schedules job/process **from ready queue to load it onto the CPU.**
   - **Dispatcher:** it is a system program which loads a process onto the CPU which is scheduled by the CPU scheduler, and **the time required for the dispatcher to stops an execution of one process and to starts an execution of another process is referred as dispatcher latency.**

SUNBEAM

Process life Cycle.

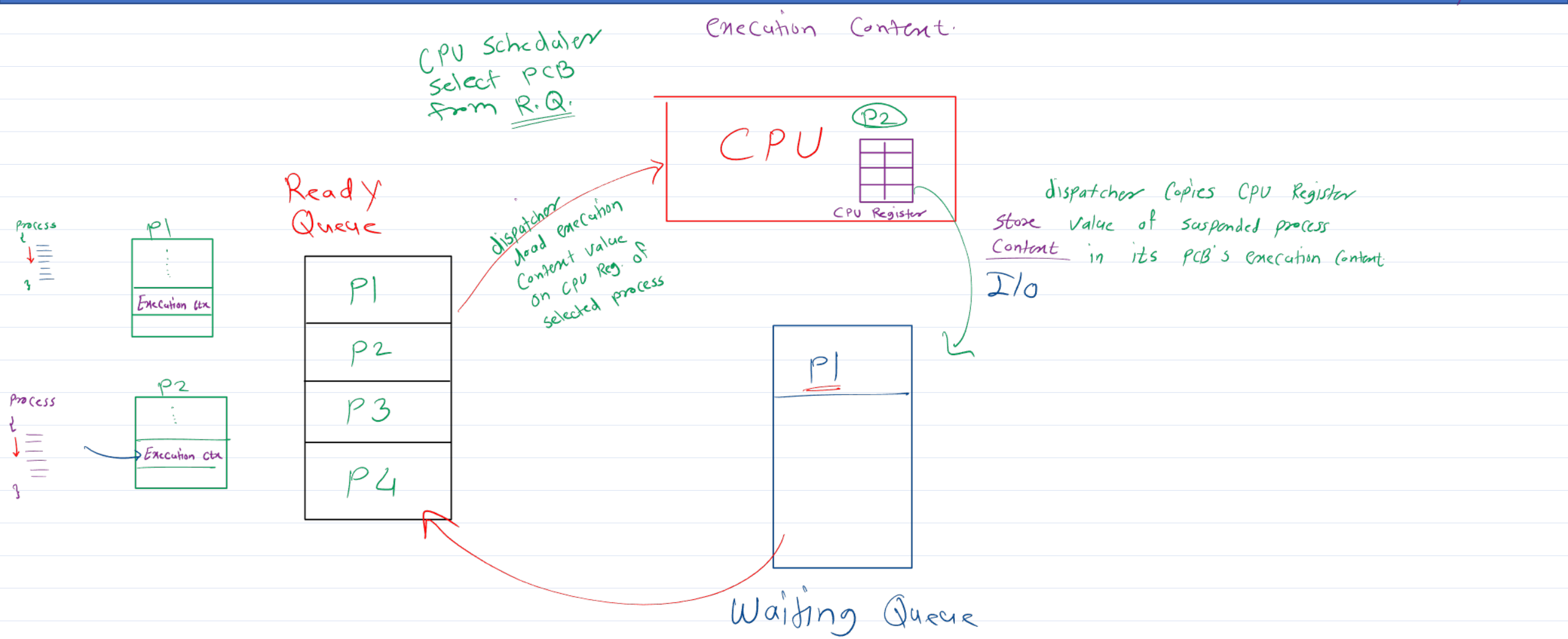


➢Context Switch:

- As during context-switch, the CPU gets switched from an execution context of one process onto an execution context of another process, and hence it is referred as **"context-switch".**
- **context-switch = state-save + state-restore**
- **state-save** of suspended process can be done i.e. an execution context of suspended process gets saved into its PCB.
- **state-restore** of a process which is scheduled by the CPU scheduler can be done by the dispatcher, dispatcher copies an execution context of process scheduled by the cpu scheduler from its PCB and restore it onto the CPU registers.
- When a high priority process arrived into the ready queue, low priority process gets suspended by means of sending an interrupt, and control of the CPU gets allocated to the high priority process, and its execution gets completed first, then low priority process can be resumed back, i.e. the CPU starts executing suspended process from the point at which it was suspended and onwards.

SUNBEAM

* Content switch => CPU switch from one process to another process



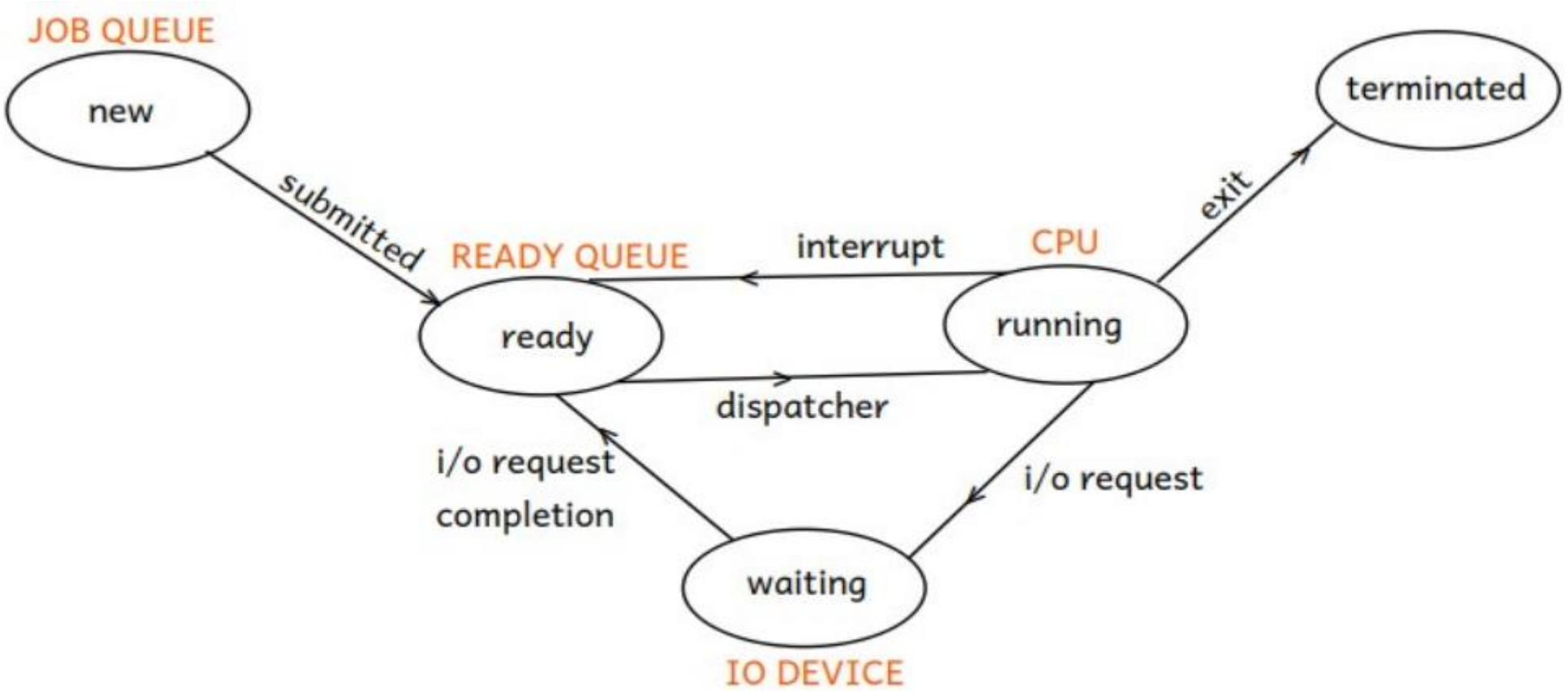
Process States:

❖Throughout execution, process goes through different states out of which at a time it can be only in a one state.

1. **New state:**
   - New process PCB is created and added into job queue. PCB is initialized and process get ready for execution.
2. **Ready state:**
   - The ready process is added into the ready queue. Scheduler pick a process for scheduling from ready queue and dispatch it on CPU
3. **Running state:**
   - The process runs on CPU. If process keeps running on CPU, the timer interrupt is used to forcibly put it into ready state and allocate CPU time to other process
4. **Waiting state:**
   - If running process request for IO device, the process waits for completion of the IO. The waiting state is also called as sleeping or blocked state
5. **Terminated state:**
   - If running process exits, it is terminated

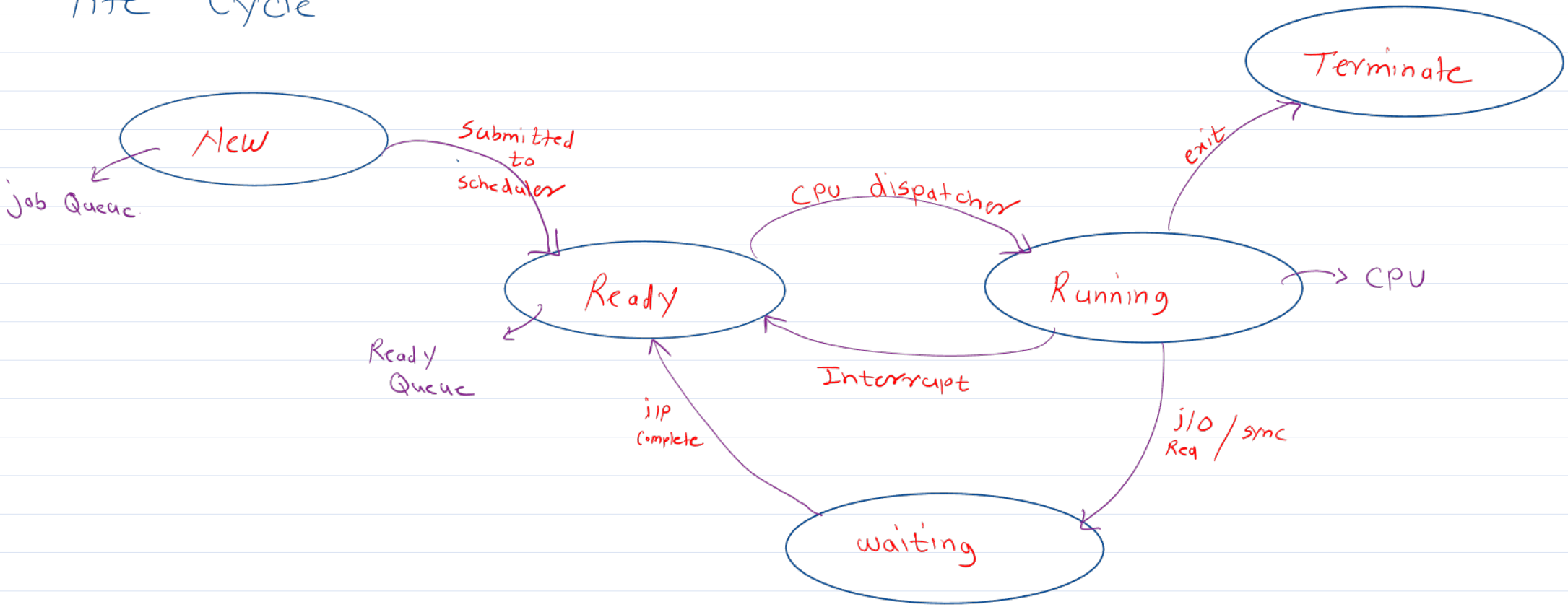
- **Linux:**
  - TASK_RUNNING (R), TASK_INTERRUPTIBLE (S), TASK_UNINTERRUPTIBLE (D), TASK_STOPPED(T), TASK_ZOMBIE (Z), TASK_DEAD (X)

SUNBEAM



PROCESS STATE DIAGRAM

* Process life Cycle



❖CPU Scheduler gets called in the following four cases:

Case-1: Running -> Terminated

Case-2: Running -> Waiting

} Non-Preemptive

Case 3: Running -> Ready

Case-4: Waiting -> Ready

} Preemptive

- There are two types of CPU scheduling:

1. **Non-preemptive:**
   - The current process gives up CPU voluntarily (for IO, terminate or yield).
   - Then CPU scheduler picks next process for the execution.
   - If each process yields CPU so that other process can get CPU for the execution, it is referred as "Co operative scheduling".

   - e.g. in above case 1 & case 2

2. **Preemptive:**
   - The current process may give up CPU voluntarily or paused forcibly (for high priority process or upon completion of its time quantum).
   - **- e.g. in above case 3 & 4.**

SUNBEAM

Following algorithms used for CPU Scheduling:

1. **FCFS (First Come First Served) CPU Scheduling**
2. **SJF (Shortest Job First) CPU Scheduling**
3. **Round Robin CPU Scheduling**
4. **Priority CPU Scheduling**

- Multiple algorithms are there for CPU scheduling, so there is need to decide which algorithm is best suited at specific situation and which algorithm is an efficient one, to decide this there are certain criteria's **called as scheduling criteria's: cpu utilization, throughput, waiting time, response time and turn-around-time.**

SUNBEAM

CPU Scheduling Criteria's:

1. CPU Utilization:

- Select such an algorithm in which utilization of the CPU must be as **maximum as a possible**.
- On server systems, CPU utilization should be more than 90%. On desktop systems, CPU utilization should around 70%.

2. Throughput:

- Total work done per unit time. One need to select such an algorithm in which throughput must be **as maximum possible.**

3. Waiting Time:

- It is the total amount of time spent by the process into the ready queue for waiting to get control of the CPU from its time of submission.
- Select such an algorithm in which waiting time must be as **minimum as possible.**

4. Response Time:

- It is a time required for the process to get first response from the CPU from its time of submission. One need to select such an algorithm in which response time must be as **minimum as possible**

SUNBEAM

5. Turn-Around -Time:

- It is the total amount of time required for the process to complete its execution from its time of submission.

- One need to select such an algorithm in which turn-around-time must be as **minimum as possible.**
- **Execution Time:** it is the **total amount of time spent by the process onto the CPU to complete its execution**.

  - OR CPU Burst Time: total no. of CPU cycles required for the process to complete its execution.

- **Turn-Around-Time = Waiting Time + Execution Time.**
- **Turn-around-time is the sum of periods spent by the process into ready queue for waiting and onto the CPU for execution from its time of submission.**

SUNBEAM

➢CPU Scheduling

1. FCFS (First Come First Served) CPU Scheduling

- In this algorithm, process which is arrived first into the ready queue gets the control of the CPU first i.e. control of the CPU gets allocated for processes as per their order of an arrival into the ready queue.
- This algorithm is simple to impalement and can be implemented by using fifo queue.
- It is a non-preemptive scheduling algorithm.

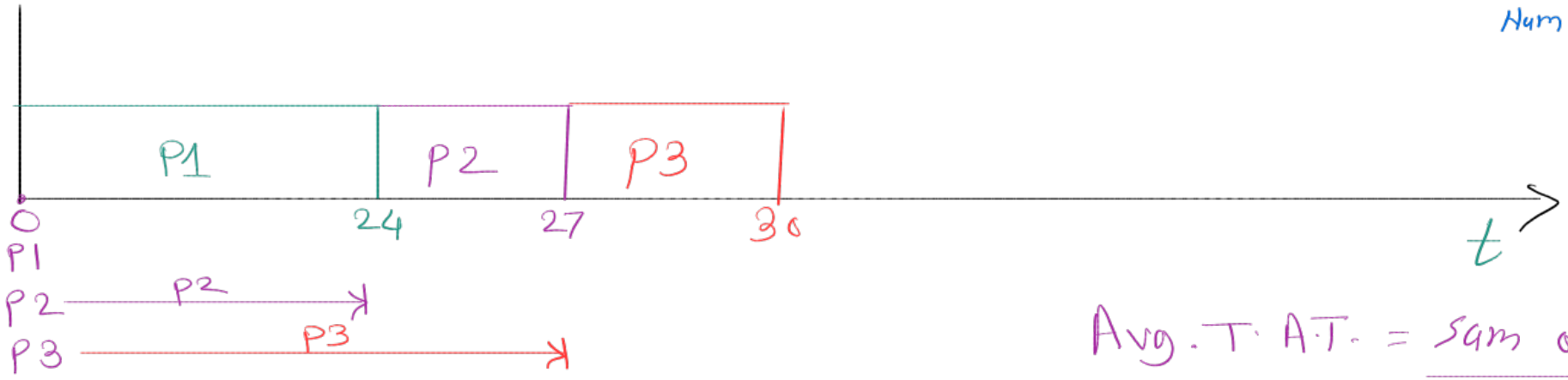
❑**Convoy effect:** in fcfs, due to an arrival of longer process before shorter processes, shorter processes has to wait for longer duration and due to which average waiting time gets increases, which results into an increase in an average turn-around-time and hence overall system performance gets down.

SUNBEAM

➢FCFS Scheduling

CPU BURST + W.T

Process	Arrival Time	CPU Burst	Wait Time	Turn Around Time
P1	0	24	0	24
P2	0	3	24	27
P3	0	3	27	30

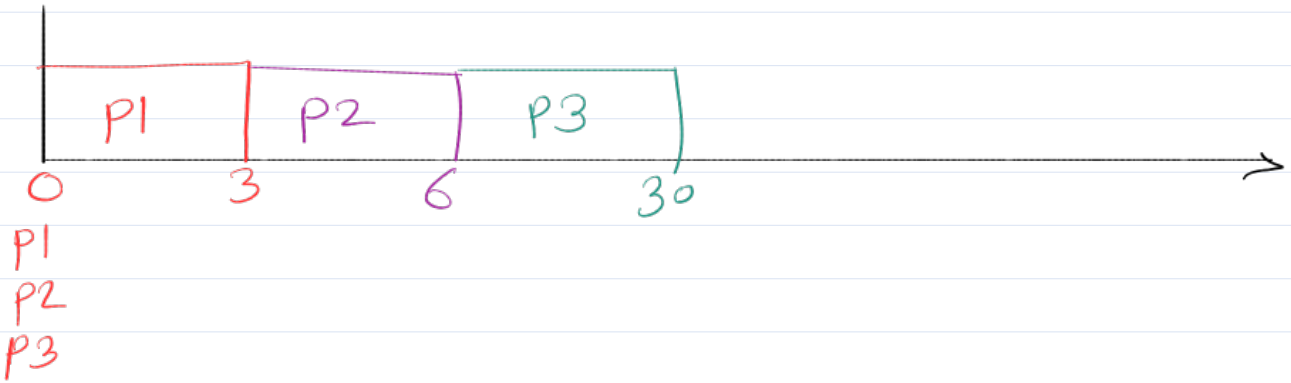


$$\text{Avg. W.T.} = \frac{\text{Sum of process W.T.}}{\text{Num of process}} = \frac{0+24+27}{3} = \frac{51}{3} \Rightarrow 17$$

$$\text{Avg. T.A.T.} = \frac{\text{Sum of all process T.A.T}}{\text{Num of processes}} = \frac{24+27+30}{3} \Rightarrow \frac{81}{3} \Rightarrow 27$$

Process	A.T.	CBT	W.T.	T.A.T
P1	0	3	0	3
P2	0	3	3	6
P3	0	24	6	30

Avg W.T. $\Rightarrow \frac{0+3+6}{3} \Rightarrow \frac{9}{3} \Rightarrow 3$



P1
P2
P3

2. SJF(Shortest Job First) CPU Scheduling:

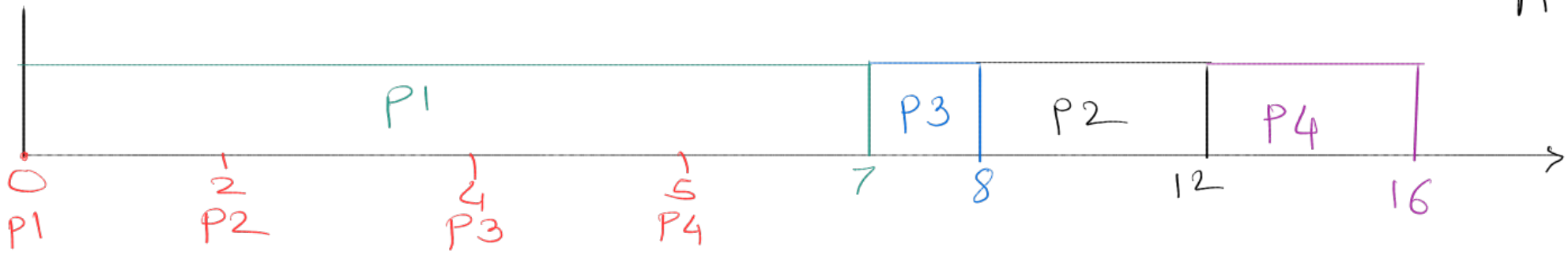
- In this algorithm, process which is having minimum CPU burst time gets the control of the CPU first, and whenever tie is there it can be resolved by using fcfs. - SJF algorithm ensures minimum waiting time.
- Under non-preemptive SJF, algorithm fails if the submission time of processes are not same, and hence it can be implemented as preemptive as well.
- Non-preemptive SJF is also called as SNTF(Shortest-Next-Time-First).
- Preemptive SJF is also called as SRTF(Shortest-Remaining-Time-First).

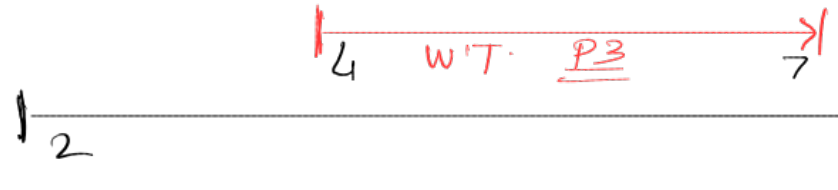
❑ **Starvation:** in this algorithm, as shorter processes has got higher priority, process which is having larger CPU burst time may gets blocked i.e. control of the CPU will never gets allocated for it, such situation is called as starvation/indefinite blocking.

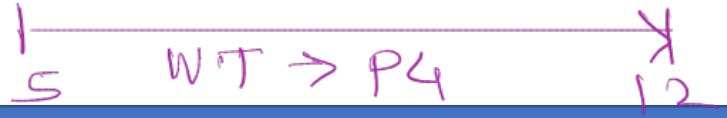
SUNBEAM

➢ SJF/SNTF Scheduling (Non - Preemptive)

Process	Arrival Time	CPU Burst	Wait Time	Turn Around Time
P1	0	7	0	7
P2	2	4	6	10
P3	4	1	3	4
P4	5	4	7	11







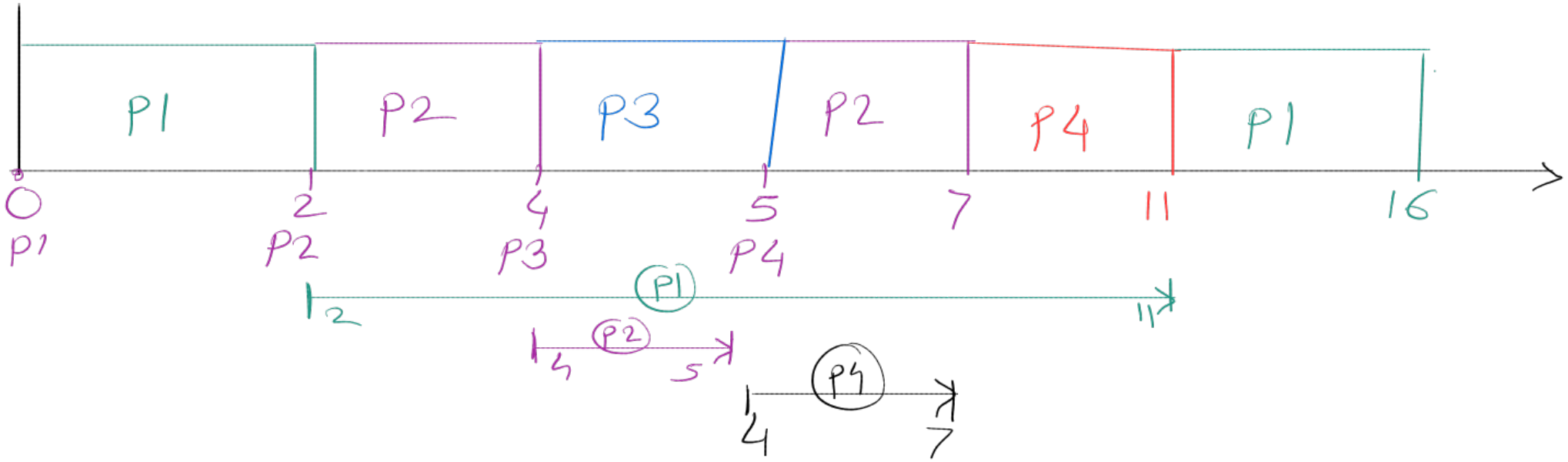
Avg. W.T $\Rightarrow \frac{0+6+3+7}{4}$

$\Rightarrow \frac{16}{4}$

$= 4$

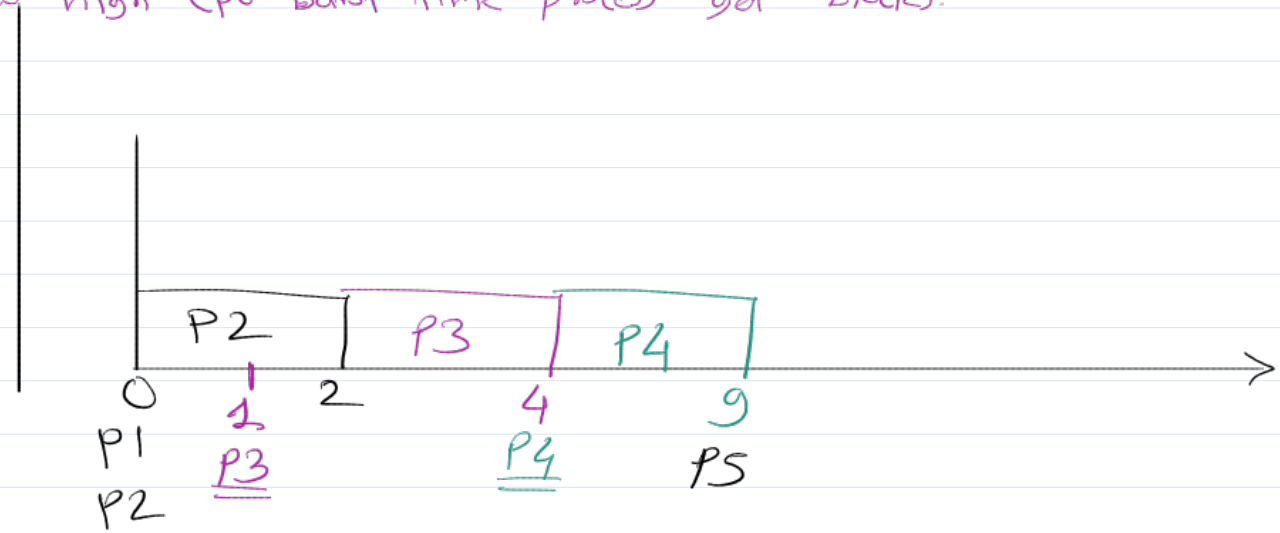
➢ SRTF Scheduling (Preemptive)

Process	Arrival Time	CPU Burst	Remaining Time	Wait Time	Turn Around Time
P1	0	7	5	9	16
P2	2	4	2 X	1	5
P3	4	1	X	0	2
P4	5	4	X	2	6



* Starvation : SJF : if low cpu burst time process comes at Runtime the low high cpu burst time process get blocks.

	Process	A.T.	C.B.T
Block (Starving)	P1	0	8
	P2	0	2 X
	P3	1	2 X
	P4	4	5 X
	P5	9	4



3. Round Robin Scheduling Algorithm

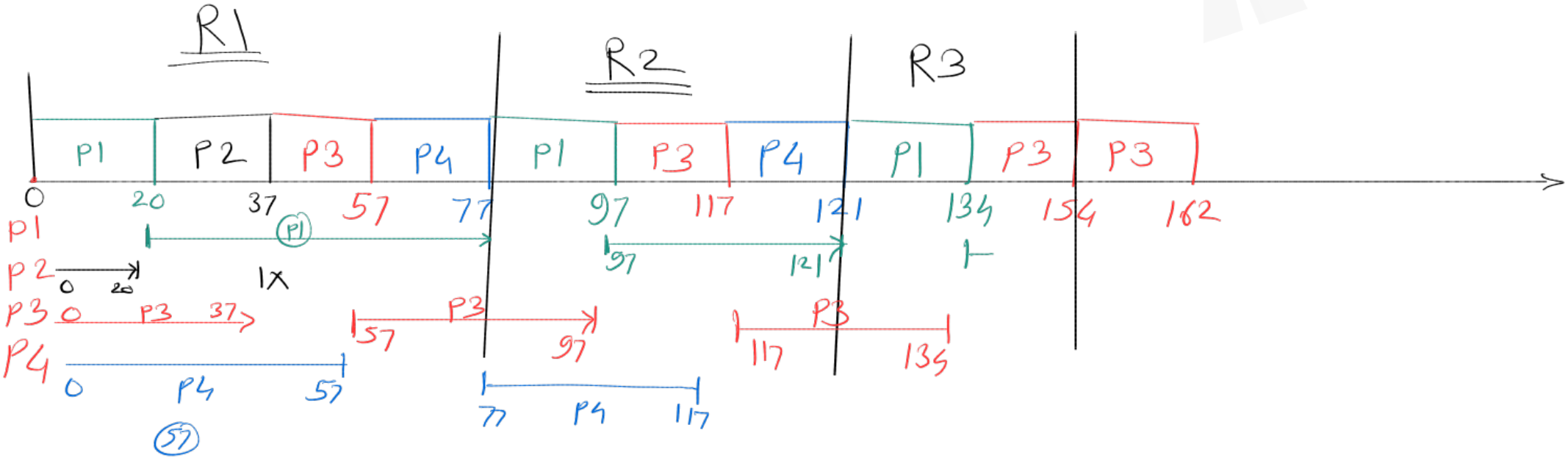
- In this algorithm, before allocating the CPU for processes, **some fixed time slice or time quantum** gets decided in advanced, and at any given time control of the CPU may remains allocated with any process maximum for that decided time-slice, once the given time slice is finished of that process, it gets suspended and control of the CPU will be allocated to the next process again for maximum that decided time slice and so on..., each process gets control of the CPU in a round robin manner i.e. cpu gets shared among processes equally.
- If any process completes its execution before allocated time slice then control of the CPU will be released by that process and CPU gets allocated to the next process as soon as it is completed for effective utilization of the CPU.
- There **is no starvation in RR Scheduling algorithm**.
- This algorithm is **purely preemptive.**
- This algorithm **ensures minimum response time**.
- **If time slice is minimum then there will be extra overhead onto the CPU due to frequent context-switch.**

SUNBEAM

➢ Round Robin Scheduling

(Time Quantum => 20)

Process	AT	CPU Burst	Remaining Time				Wait Time	Response Time
P1	0	53	33	13	X	X	57+24 => 81	0
P2	0	17	X	X	X	X	20	20
P3	0	68	48	28	8	X	37+40+17 => 94	37
P4	0	24	4	X	X	X	57+40 => 97	57
			R1	R2	R3			


R1
R2
R3
P1
P2
P3
P4
P1
P3
P4
P1
P3
P3
0
20
37
57
77
97
117
121
134
154
162
P1
P2
P3
P4
P3
P3
P4
P4
P3
P4
P3
P4
P3

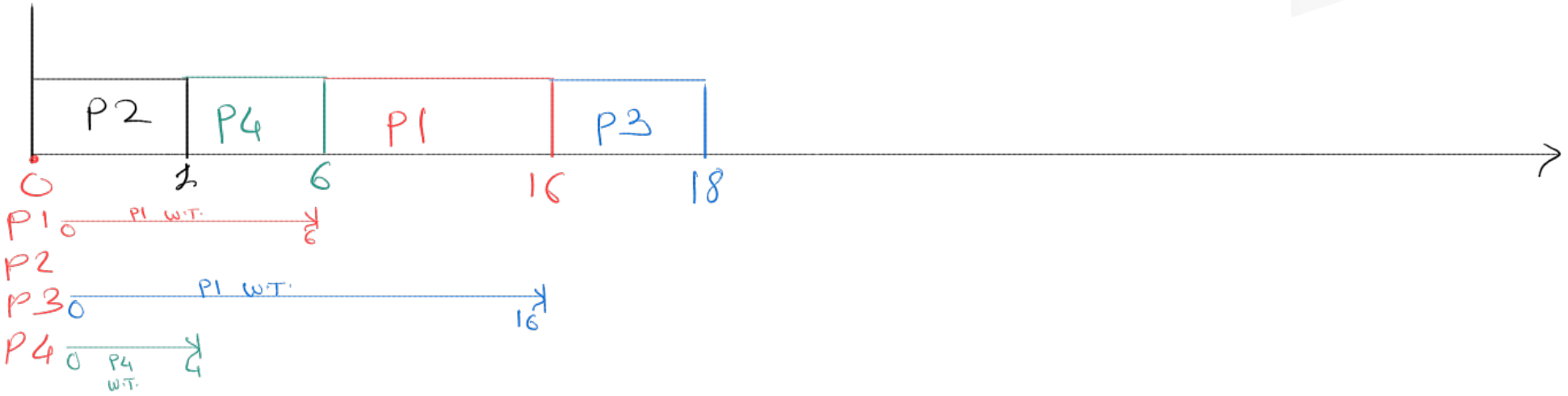
4. Priority Scheduling

- In this algorithm, process which is having highest priority gets control of the CPU first**, each process is having priority in its PCB**.
- priority for a process can be decided by two ways:
    1. **internally** – priority for process can be decided by an OS depends on no. of resources required for it.
    2. **externally** – priority for process can be decided by the user depends on requirement.
- **Minimum priority value indicates highest priority.**
- This algorithm **is purely preemptive.**
- **Due to the very low priority process may gets blocked into the ready queue and control of the CPU will never gets allocated for such a process, this situation is referred as a starvation or indefinite blocking**.
- **Ageing:** it is a technique in which, an OS gradually increments priority of blocked process, i.e. priority of blocked process gets incremented after some fixed time interval by an OS, so that priority of blocked process becomes sufficient enough to get control of the CPU, and starvation can be avoided.

SUNBEAM

➢ Priority Scheduling

Process	Arrival Time	CPU Burst	Priority	Wait Time
P1	0	10	3	6
P2	0	1	1	0
P3	0	2	4	16
P4	0	5	2	1



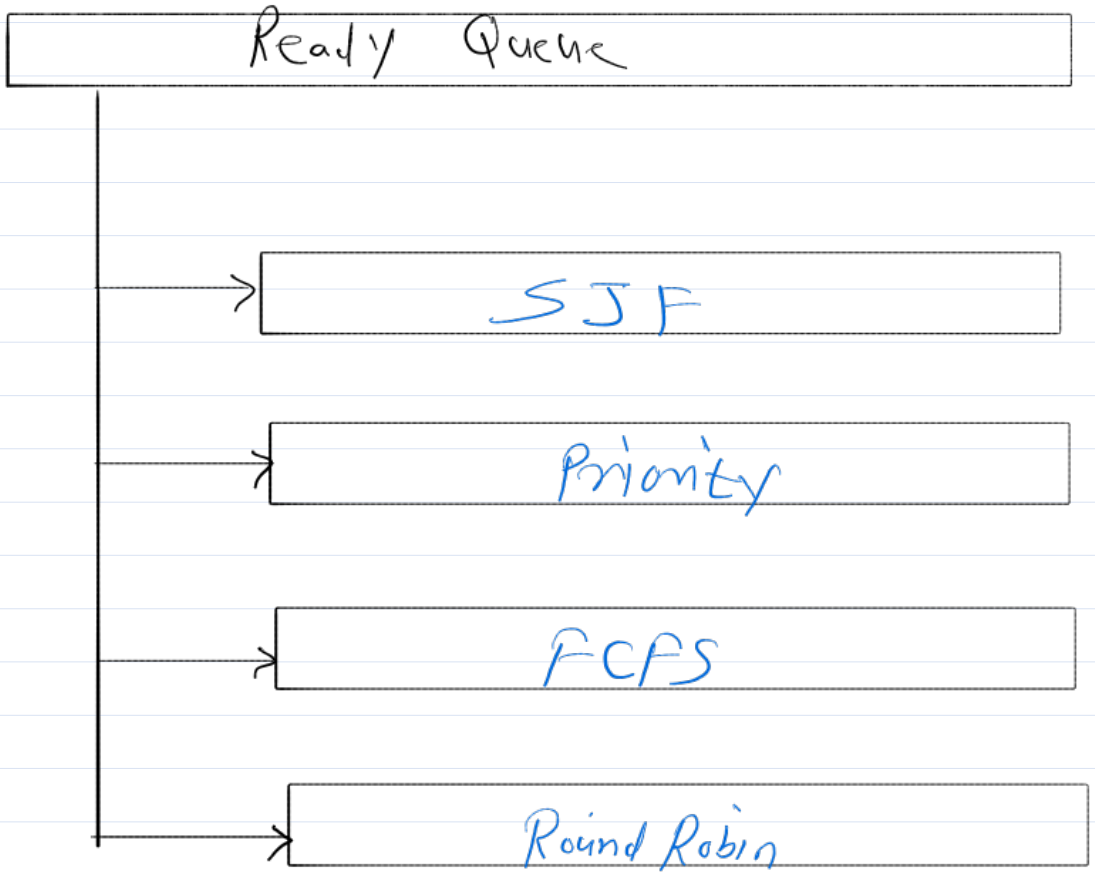
➢Multi-level queue :

- In modern OS, the ready queue can be divided into multiple sub-queues and processes are arranged in them depending on their scheduling requirements. This structure is called as "Multi-level queue".
- If a process is starving in some sub-queue due to scheduling algorithm, it may be shifted into another sub-queue. This modification is referred as "Multi-level feedback queue".
- The division of processes into sub - queues may differ from OS to OS.

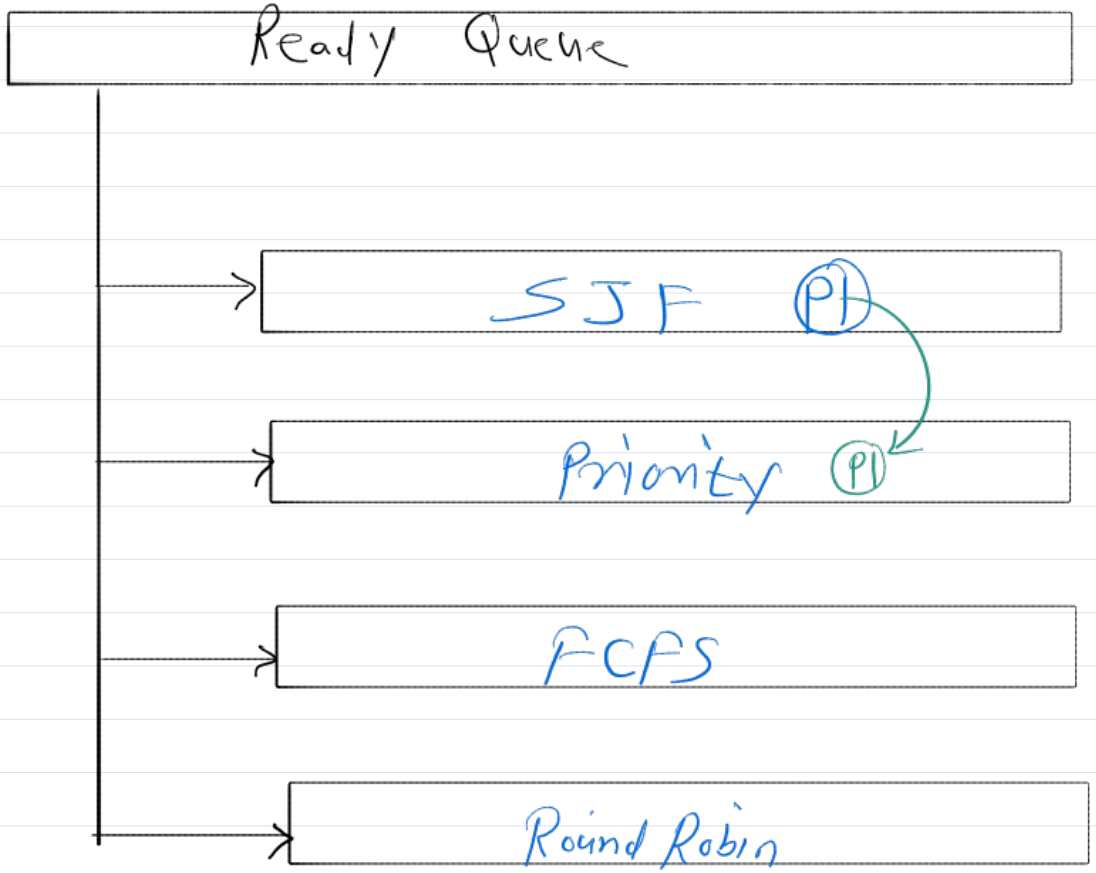
1. Important Services :Priority Scheduling
2. Background Task :SJF
3. GUI Tasks :RR
4. Other Tasks :FCFS

SUNBEAM

Maltilevel Queue



Maltilevel F/b Queue





Thank you!

Kiran Jaybhave

email – kiran.jaybhave@sunbeaminfo.com